



Microservice Patterns: Complete Reference

System Design Patterns for Modern Distributed Systems

2.

Agenda



1. Foundations and Design Strategies
2. Communication, Management, and Resilience
3. Deployment, Observability, and Architecture

3.



Foundations and Design Strategies

4.

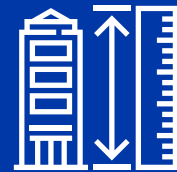


Introduction to Microservice Patterns



Addresses Distributed System Challenges

Microservice patterns tackle issues like scalability and resilience.



Provides Reusable Architectural Solutions

Patterns offer proven strategies for system design and structure.



Enables Scalable, Resilient Applications

Understanding patterns is essential for building robust systems.

5.



Service Decomposition Strategies



Decompose by Business Capability

Assign each service a distinct business function for clear ownership.



Subdomain Decomposition

Use strong domain boundaries to organize services by bounded context.



Avoid Verb-Based Decomposition

Grouping by actions leads to tight coupling and should be avoided.

6.



Managing Data Across Services

Service-Owned Databases

Each microservice manages its own data store for independence.

API Composition & CQRS

Combine data from multiple services; separate read/write models for scalability.

Event Sourcing

Persist state changes as events for auditability and recovery.



7.



Communication, Management, and Resilience

8.

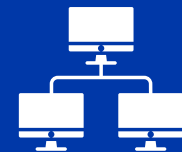


Communication and Consistency



Synchronous vs Asynchronous Messaging

REST/gRPC for direct calls;
messaging for decoupling



Event-Driven Architecture

Services communicate via events
for scalability



Distributed Transaction Management

Saga and eventual consistency
patterns ensure reliability



API Management and Service Discovery

API Gateway: Centralized Access & Security

- Acts as a single entry point for all client requests
- Handles authentication, routing, and rate limiting

Backend for Frontend (BFF): Client-Specific APIs

- Customizes API responses for web, mobile, or admin clients
- Improves client experience and reduces complexity

Service Discovery: Reliable Inter-Service Communication

- Enables dynamic location of service instances
- Supports scaling and fault tolerance in distributed systems



Resilience and Fault Tolerance

Preventing Cascading Failures

- Circuit breaker, retry, and timeout patterns stop error propagation.

Isolating and Protecting Services

- Bulkhead pattern isolates failures to protect critical components.
- Rate limiting controls incoming requests to prevent overload.

Building Resilient Microservices

- Combining these patterns ensures high availability and reliability.

11.



Deployment, Observability, and Architecture

12.



Deployment and Security Approaches

Flexible Deployment Strategies

Use containers, serverless, or sidecar patterns for service deployment.

Safe Release Techniques

Blue-green and canary deployments minimize risk during rollouts.

Robust Security Patterns

Implement API keys, OAuth, mTLS, and zero trust for secure access.



Observability and Migration

Operational Visibility with Observability

- Externalized configuration enables dynamic updates without redeploying services.
- Centralized logging aggregates logs for easier troubleshooting and monitoring.

Enhanced Monitoring and Diagnostics

- Distributed tracing tracks requests across service boundaries for root cause analysis.
- Health checks provide real-time status for service availability and readiness.

Seamless Migration Strategies

- Strangler Fig pattern allows gradual replacement of monolith functionality.
- Anti-corruption layer protects new microservices from legacy system complexities.

14.



Code Structure and Key Takeaways



Maintainable Code with Hexagonal & Clean Architecture

Promotes separation of concerns and testability.



Modular Monoliths as Microservice Foundations

Enable gradual transition to distributed systems.



Pattern Choice Drives Scalability & Security

Selecting the right patterns ensures resilient systems.